

Estudo de arquiteturas de CNN no dataset *CIFAR-10*

Gabriel Brum

4 de maio de 2026

Conteúdo

1	Introdução	2
1.1	CNN-A: A menor arquitetura	2
1.2	CNN-B: A arquitetura média	2
1.3	CNN-C: A arquitetura grande	4
2	Resultados	5
2.1	CNN-A	5
2.1.1	Acurácia, Loss e Predições	5
2.1.2	Filtros e <i>Feature Maps</i>	5
2.2	CNN-B	5
2.2.1	Acurácia, Loss e Predições	5
2.2.2	Filtros e <i>Feature Maps</i>	8
2.3	CNN-C	8
2.3.1	Acurácia, Loss e Predições	8
2.3.2	Filtros e <i>Feature Maps</i>	9
3	Conclusão	9

1 Introdução

Esta seção expõe as topologias escolhidas, os hiperparâmetros selecionados e suas respectivas justificativas.

O código foi desenvolvido em Python, executado no servidor do grupo de pesquisa LIPAI, e está disponível no *github* [].

1.1 CNN-A: A menor arquitetura

```
1 "CNN_A": {
2     "conv_configs": [
3         {"filters": 32, "kernel": 3},      # conv1: 32 filtros 3x3
4         {"filters": 64, "kernel": 3},      # conv2: 64 filtros 3x3
5     ],
6     "fc_sizes"      : [256, 128],
7     "dropout_p"     : 0.0,
8     "optimizer"     : "adam",
9     "lr"             : 1e-3,
10 }
```

Listing 1: Arquitetura A (CNN simples).

Esta arquitetura visa ser a *baseline*, onde não realizamos regularização, assim espera-se que haja *overfitting* e que a acurácia de validação seja inferior às demais arquiteturas. Em *Listing 1*, está o código da arquitetura CNN-A, que consiste em duas camadas convolucionais seguidas por duas camadas totalmente conectadas (fully connected). As camadas convolucionais utilizam filtros de 3x3, com 32 e 64 filtros respectivamente, e são seguidas por camadas de pooling para reduzir a dimensionalidade. As camadas totalmente conectadas possuem 256 e 128 neurônios, respectivamente, e não utilizam dropout para regularização. O otimizador escolhido é o Adam, com uma taxa de aprendizado de 0.001. Para melhor visualização, o esquema presente na Figura 1 mostra o fluxo entre as camadas.

1.2 CNN-B: A arquitetura média

```
1 "CNN_B": {
2     "conv_configs": [
3         {"filters": 64, "kernel": 5},      # conv1: 64 filtros 5x5
4         {"filters": 128, "kernel": 3},      # conv2: 128 filtros
5         3x3                                # conv3: 128 filtros
6         {"filters": 128, "kernel": 3},      3x3
7     ],
8     "fc_sizes"      : [200, 200],
9     "dropout_p"     : 0.1,
10    "optimizer"     : "adam",
11    "lr"             : 1e-3,
12 }
```

Listing 2: Arquitetura B (CNN média).

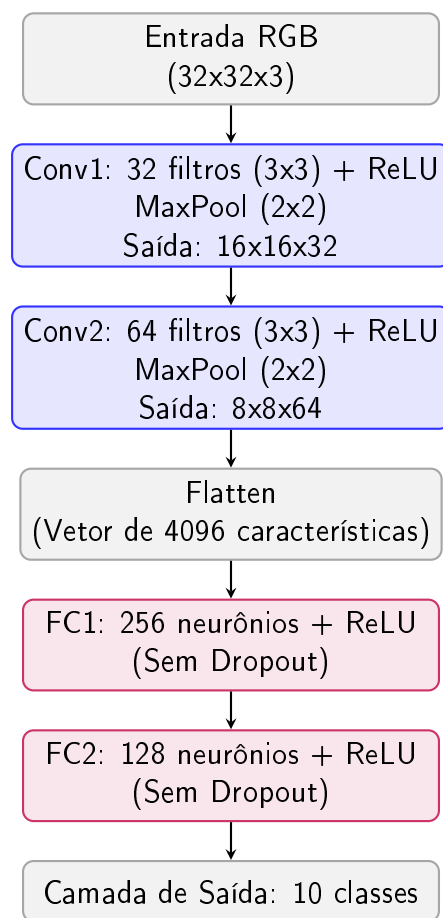


Figura 1: Arquitetura CNN-A: Baseline Simples (Otimizador: Adam, LR: 1e-3, Dropout: 0.0)

A arquitetura CNN-B é uma versão intermediária, com mais camadas e regularização leve. Como exposto em *Listing 2* ela possui três camadas convolucionais, com 64 filtros de 5x5 na primeira camada e 128 filtros de 3x3 nas duas camadas seguintes. As camadas totalmente conectadas têm 200 neurônios cada, e um dropout de 0.1 é aplicado para reduzir o risco de overfitting. O otimizador e a taxa de aprendizado são os mesmos da CNN-A. A Figura 2 ilustra a estrutura da CNN-B, destacando as diferenças em relação à CNN-A, como o aumento no número de camadas e a introdução do dropout.

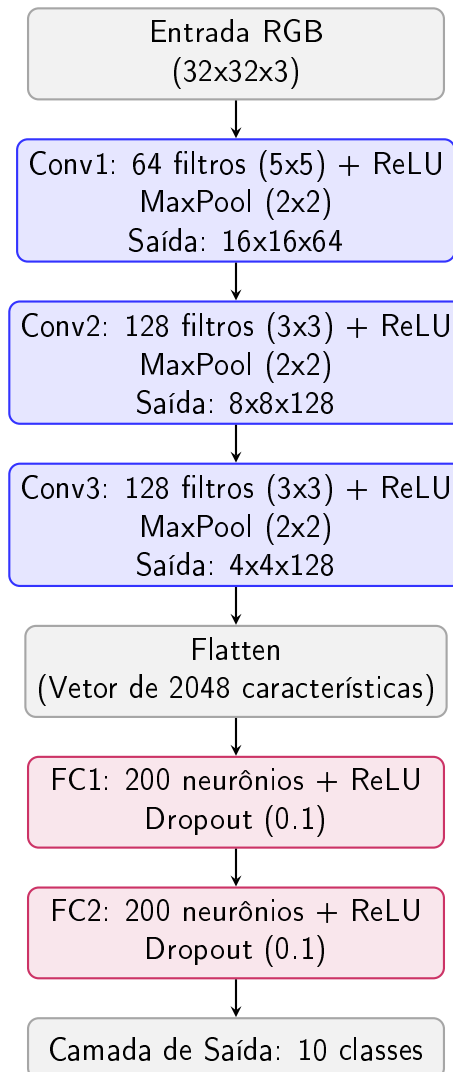


Figura 2: Arquitetura CNN-B: Média (Otimizador: Adam, LR: 1e-3, Dropout: 0.1)

1.3 CNN-C: A arquitetura grande

```

1 "CNN_C": {
2     "conv_configs": [
3         {"filters": 32, "kernel": 3},
4         {"filters": 64, "kernel": 3},
5         {"filters": 128, "kernel": 3},
6         {"filters": 128, "kernel": 3},
7     ],

```

```

8         "fc_sizes"      : [256, 128],
9         "dropout_p"     : 0.3,
10        "optimizer"     : "sgd",
11        "lr"             : 1e-2,
12    }

```

Listing 3: Arquitetura C (CNN grande).

Aqui a arquitetura é mais robusta, com quatro camadas convolucionais e uma regularização mais forte. Conforme mostrado em *Listing 3*, as camadas convolucionais têm 32, 64, 128 e 128 filtros de 3×3 , respectivamente. As camadas totalmente conectadas possuem 256 e 128 neurônios, com um dropout de 0.3 para combater o overfitting. O otimizador escolhido é o SGD, que geralmente requer uma taxa de aprendizado maior ($1e-2$) em comparação ao Adam. A Figura 3 apresenta a estrutura da CNN-C, evidenciando o aumento no número de camadas e a mudança no otimizador, que pode impactar significativamente o processo de treinamento e a convergência do modelo.

2 Resultados

2.1 CNN-A

2.1.1 Acurácia, Loss e Predições

A arquitetura CNN-A atingiu uma acurácia de teste de 77.38%. O critério de parada antecipada (*early stopping*) foi acionado na época 25, indicando que o modelo começou a saturar e perder capacidade de generalização precocemente. Apesar da ausência de camadas de *dropout*, a aplicação do *Data Augmentation* no pré-processamento das imagens mitigou o sobreajuste severo. A Figura 4 ilustra o histórico de treinamento, onde a função de custo (*loss*) de validação estabiliza na faixa de 0.69, não acompanhando a queda contínua da perda de treinamento. A Figura 5 apresenta uma amostra das predições realizadas no conjunto de teste.

2.1.2 Filtros e *Feature Maps*

A análise da primeira camada convolucional (Figura 6) revela que os filtros 3×3 aprenderam a detectar padrões de baixo nível espacial, consistindo principalmente em detectores de bordas simples e transições abruptas de cor. Os mapas de ativação (Figura 7), extraídos da última camada convolucional, evidenciam a captação das silhuetas dos objetos, porém com características topológicas primárias, compatíveis com a menor profundidade da rede.

2.2 CNN-B

2.2.1 Acurácia, Loss e Predições

A CNN-B apresentou o melhor desempenho da bateria de testes, alcançando uma acurácia de 83.00%. O treinamento prosseguiu até a época 47, denotando um aprendizado prolongado e resiliente. O uso de uma janela perceptiva maior no início (5×5) em conjunto com um *dropout* leve de 10% propiciou uma evolução suave. Observa-se na Figura 8 que a distância (*gap*) entre as curvas de validação e treino manteve-se estreita e sob controle na maior parte do processamento. A Figura 9 exemplifica as saídas na fase de predição.



Figura 3: Arquitetura CNN-C: Profunda (Otimizador: SGD, LR: 1e-2, Dropout: 0.3)

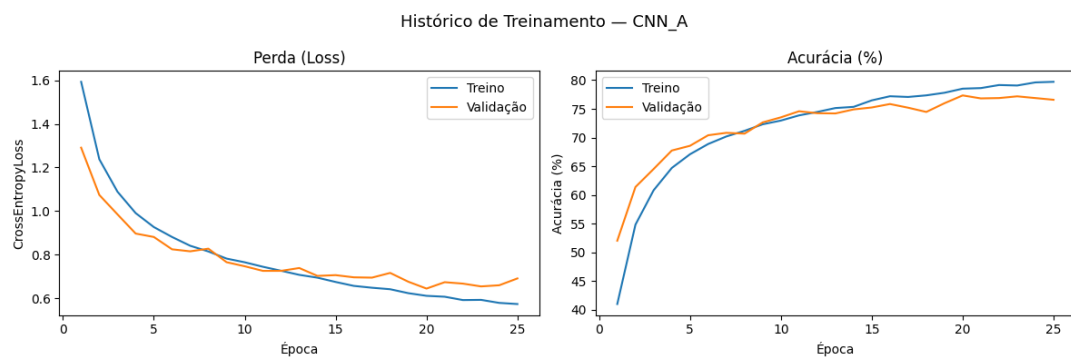


Figura 4: Histórico de *Loss* e Acurácia para a CNN-A.

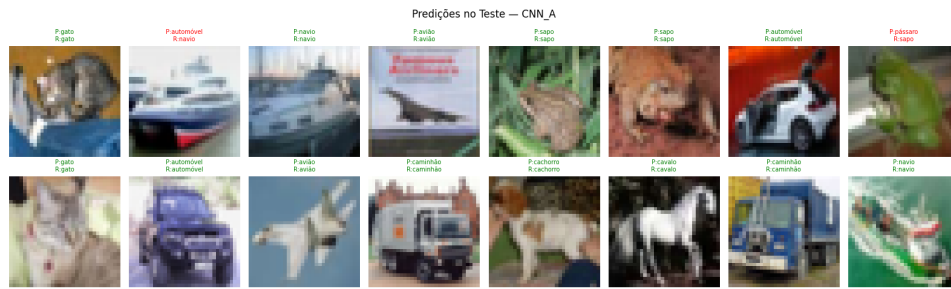


Figura 5: Amostra de predições da arquitetura CNN-A no conjunto de teste.

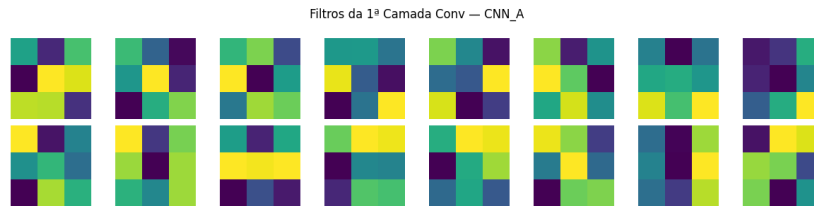


Figura 6: Filtros aprendidos na primeira camada convolucional da CNN-A.



Figura 7: Mapas de ativação (última camada convolucional) da CNN-A.

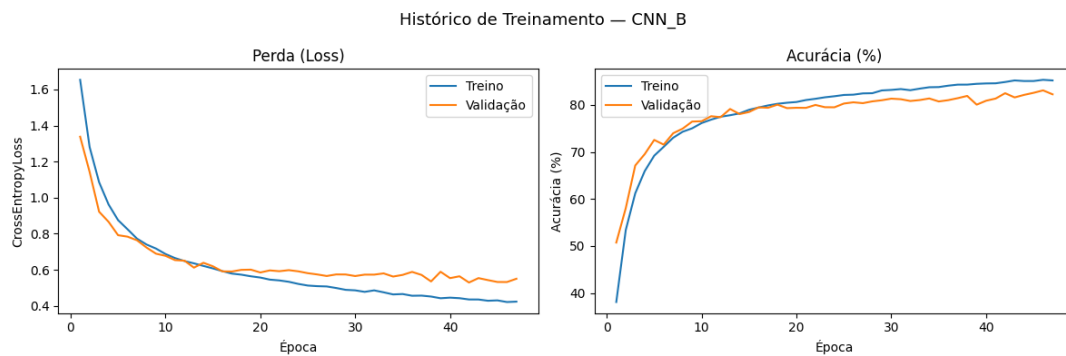


Figura 8: Histórico de *Loss* e Acurácia para a CNN-B.

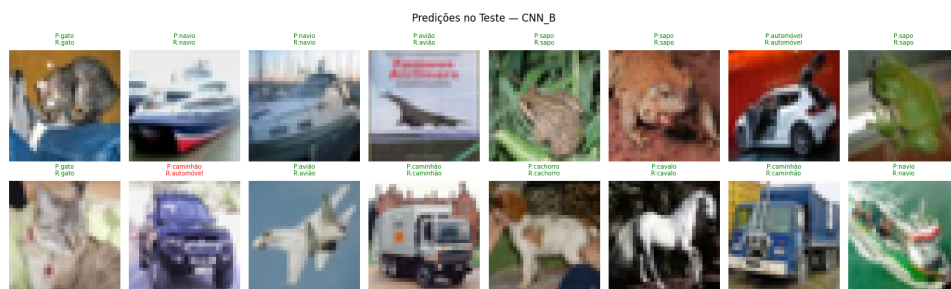


Figura 9: Amostra de predições da arquitetura CNN-B no conjunto de teste.

2.2.2 Filtros e *Feature Maps*

Os kernels maiores da camada inicial (Figura 10) permitiram a captura englobante de texturas variadas e gradientes de fundo de forma mais expressiva que a arquitetura predecessora. Os *feature maps* resultantes (Figura 11) ilustram um isolamento notório das características de interesse (e.g., contornos de carros ou focinhos), reduzindo significativamente o nível de ruído propagado para a camada de predição linear.

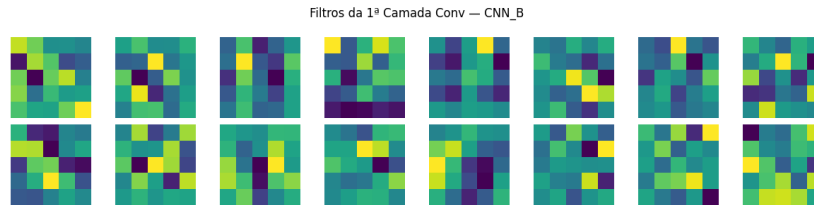


Figura 10: Filtros aprendidos na primeira camada convolucional da CNN-B.



Figura 11: Mapas de ativação (última camada convolucional) da CNN-B.

2.3 CNN-C

2.3.1 Acurácia, Loss e Predições

Tratando-se do modelo mais profundo da análise (4 camadas convolucionais), a CNN-C finalizou a rotina com 82.23% de acurácia de teste, parando o processo antecipadamente na época 34. A configuração aliada ao otimizador *SGD* e uma forte taxa de *dropout* (0.3) provocou um comportamento oscilatório acentuado ao longo das épocas (Figura 12). Todavia, a capacidade da rede de manter uma correlação íntima entre o conjunto de treinamento e de validação reforça sua alta tolerância contra a memorização (*overfitting*).

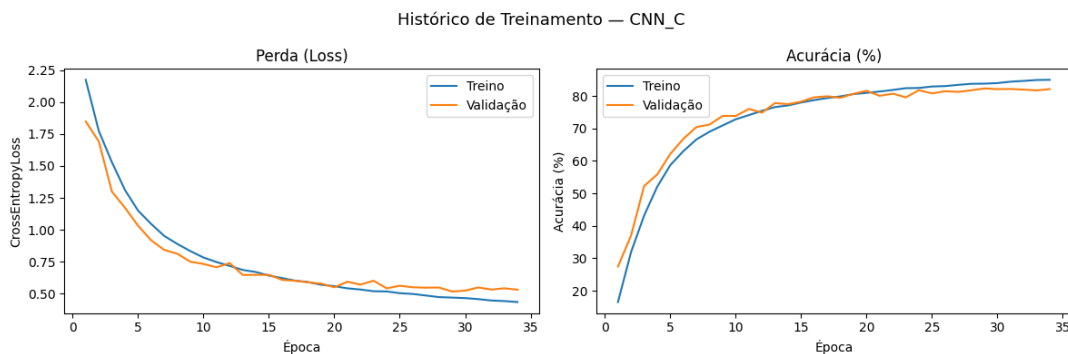


Figura 12: Histórico de *Loss* e Acurácia para a CNN-C.

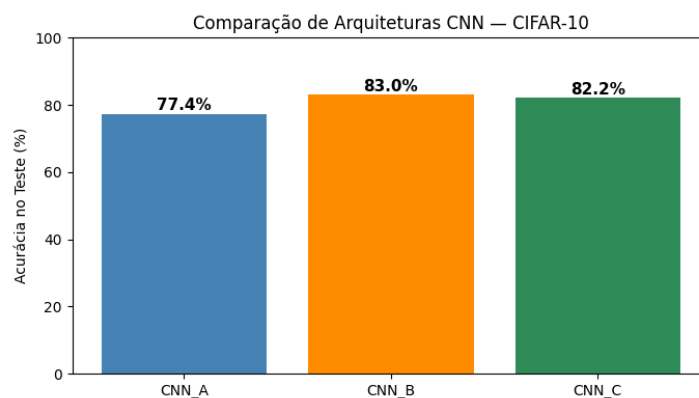


Figura 16: Comparação final da acurácia de teste entre as três arquiteturas avaliadas.

consagra empíricamente como o ponto de equilíbrio ideal no contexto destas dimensões de imagem (32×32); operando com filtros estendidos na captação inicial e uma regularização contida de *dropout*, sem desestabilizar os pesos com o passar das épocas. Por fim, valida-se o impacto crítico do pipeline de *Data Augmentation*, sendo vital como mecanismo contínuo de imunização das redes contra o fenômeno perene do sobreajuste.